

Manual de usuario

Enigma

Laboratorio de razonamiento

Cómo instalar, usar y sacarle provecho al lenguaje y al IDE

Paradigmas de Programación · EIF-400

Universidad Nacional de Costa Rica
Sede Regional Brunca · Campus Pérez Zeledón

Estudiantes

Alisson Cordero Fonseca
Melany Céspedes Sánchez

Profesor: Pablo Venegas Elizondo

II Ciclo · 2026

Contenido

1. ¿Qué es Enigma? 3
2. Instalación y arranque 3
2.1 Qué necesitas 3
2.2 Abrir el IDE 3
3. Recorrido por el IDE 4
4. Trabajar con proyectos 5
4.1 Abrir un proyecto que ya existe 5
4.2 Crear un proyecto y su primer archivo 5
4.3 Renombrar y eliminar 5
4.4 El archivo principal 5
5. Tu primer programa 5
6. El lenguaje en pocas páginas 6
6.1 Lo básico 7
6.2 Tipos de datos 7
6.3 Decisiones y ciclos 7
6.4 Escenas y procesos (funciones) 8
6.5 Operaciones 8
6.6 Reglas que conviene recordar 8
7. Compilar y ejecutar 9
7.1 Pedir datos con consultar 9
7.2 Salida y Terminal 9
7.3 Atajos de teclado 9
8. Entender los mensajes 10
8.1 Los dos pasos de un error 10
8.2 Tipos de error 11
8.3 Advertencias 12
9. Importar funciones entre archivos 12
10. Las ayudas del IDE 13
10.1 La Guía del detective 13
10.2 Casos de práctica y acertijos 15
10.3 El Constructor de bloques 16
10.4 Buscar y preferencias 16
11. Proyectos de ejemplo 17
12. Solución de problemas 17

1. ¿Qué es Enigma?

Enigma es un lenguaje de programación educativo con su propio entorno gráfico, un IDE donde escribes, compilas y ejecutas tus programas. Está pensado para entrenar el **razonamiento lógico**: cada programa es un caso que se resuelve deduciendo, y cuando algo sale mal el IDE no te da la respuesta de inmediato. Primero te entrega un **indicio** para que busques el error tú mismo y, si lo necesitas, después te explica con detalle qué falló.

Los programas se guardan en archivos con extensión `.eng` y toda la sintaxis usa el vocabulario de una investigación detectivesca:

Palabra	Qué significa	En otros lenguajes
`pista`	Una variable: guarda un dato que puede cambiar.	variable
`hecho`	Una constante: un dato que no puede cambiar.	constante
`caso`	El programa principal, el punto donde empieza todo.	main
`escena`	Una función que devuelve un valor.	función con retorno
`proceso`	Una función que no devuelve nada.	procedimiento
indicio	El primer aviso de un error: te orienta sin decir dónde está.	mensaje de error

2. Instalación y arranque

2.1 Qué necesitas

- **Python 3.10 o superior.**
- **tkinter**, la librería de ventanas. Ya viene incluida con Python en Windows y macOS. En Linux, si el IDE no abre, instálala con `sudo apt install python3-tk`.
- **pytest** solo si quieres correr las pruebas del proyecto (no hace falta para usar el IDE).

2.2 Abrir el IDE

Abre una terminal en la carpeta del proyecto (la que contiene `main.py`) y ejecuta:

```
python main.py                                  # abre el IDE vacío
python main.py ejemplos/investigacion_caso      # abre el IDE con un proyecto de ejemplo
```

Para instalar la dependencia de las pruebas (opcional):

```
pip install -r requirements.txt
```

3. Recorrido por el IDE

Esta es la ventana principal de Enigma. Cada número de la figura corresponde a una parte descrita en la tabla.

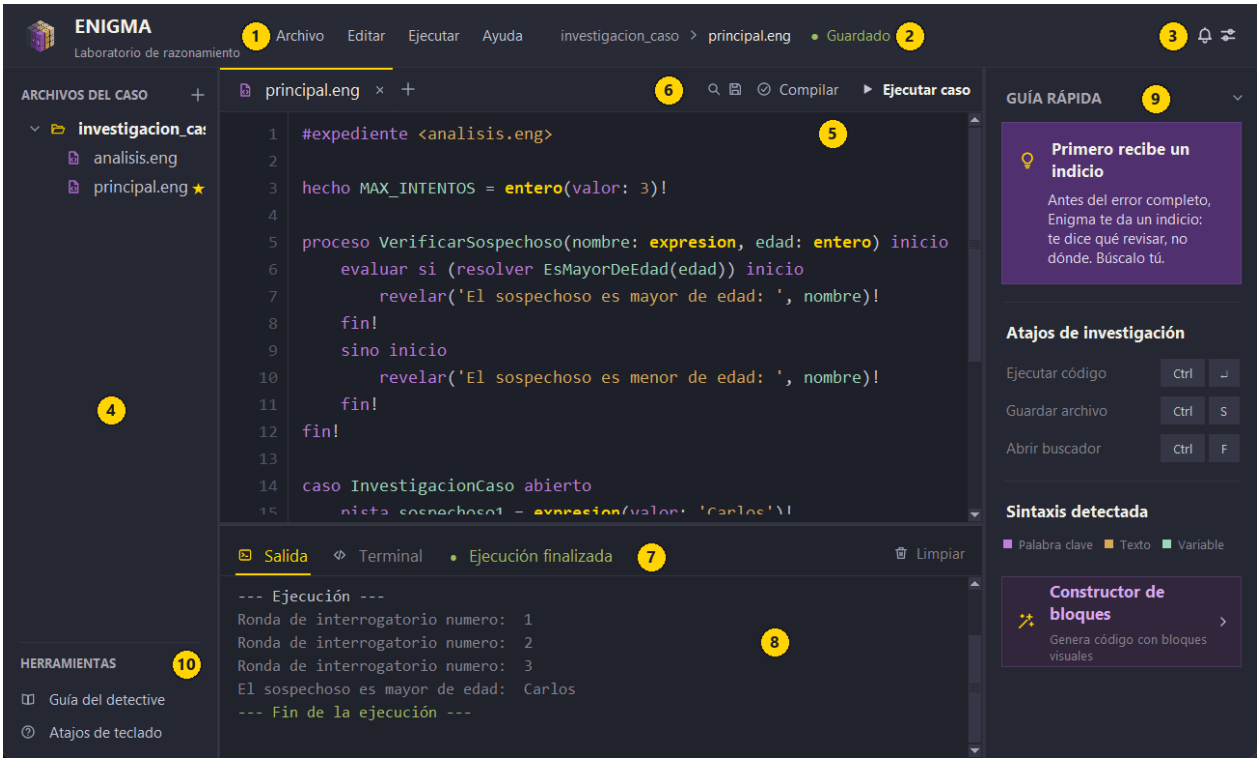


Figura 1. Ventana principal del IDE con sus partes numeradas.

N.º	Parte	Para qué sirve
1	Menús	**Archivo** (abrir carpeta, guardar, cerrar pestaña), **Editar** (buscar, preferencias), **Ejecutar** (compilar y ejecutar) y **Ayuda** (guías y constructor).
2	Archivo activo	Muestra el proyecto y el archivo que estás editando, y si tiene cambios sin guardar.
3	Historial y preferencias	La campana abre la bitácora de la sesión y se marca cuando hubo un error. El ícono de ajustes cambia el tamaño de la letra.
4	Árbol de archivos	Los archivos y carpetas del proyecto. La ★ marca el archivo principal y el botón **+** crea archivos y carpetas.
5	Editor	Pestañas por archivo, números de línea y colores para las palabras del lenguaje.
6	Compilar y Ejecutar caso	Los dos botones principales. También están la lupa (buscar) y el disquete (guardar).
7	Estado	Dice en qué paso va el IDE: Listo, Compilación exitosa, Compilación fallida, Ejecución finalizada…
8	Panel de salida	Aquí aparecen los resultados, los errores y las advertencias. La solapa **Terminal** guarda la bitácora de lo que hiciste.
9	Panel derecho	Alterna entre la **Guía rápida**, la **Guía del detective** y el **Constructor de bloques**.
10	Herramientas	Atajos para abrir la Guía del detective y la lista de atajos de teclado.

4. Trabajar con proyectos

Un **proyecto** es una carpeta con uno o más archivos `.eng`. Por ejemplo:

```
MiProyecto/
    principal.eng
    operaciones.eng
    utilidades.eng
```

4.1 Abrir un proyecto que ya existe

- Menú **Archivo → Abrir carpeta…** (o `Ctrl+O`).
- El botón **Abrir carpeta…** que aparece en el árbol cuando está vacío.
- El botón **+** del árbol y la opción **Abrir carpeta…**.

El árbol puede mostrar varios proyectos abiertos a la vez.

4.2 Crear un proyecto y su primer archivo

1. Pulsa el botón **+** junto a «ARCHIVOS DEL CASO» y elige **Nuevo archivo**.
2. Si todavía no hay ningún proyecto abierto, el IDE te pide el nombre de la carpeta del proyecto y dónde guardarla (propone la carpeta Documentos/Enigma).
3. Escribe el nombre del archivo **sin extensión**: Enigma le agrega `.eng` por ti.

Con el mismo botón **+** puedes crear **Nueva carpeta** para ordenar tus archivos.

4.3 Renombrar y eliminar

Haz **clic derecho** sobre un archivo o una carpeta del árbol:

Sobre…	Opciones del menú
Un archivo	Abrir · Renombrar · Eliminar · **Marcar como archivo principal**
Una carpeta	Nuevo archivo aquí · Nueva carpeta aquí · Renombrar · Eliminar carpeta

4.4 El archivo principal

Todo proyecto necesita un **archivo principal**: el punto de entrada por donde empieza la ejecución. Es el que se compila y se ejecuta cuando pulsas los botones, y se reconoce por la ★ junto a su nombre.

> **Importante.** Un archivo nuevo **no se marca solo** como principal. Haz clic derecho sobre él y elige **Marcar como archivo principal**. Si no lo haces, al compilar verás el aviso «No hay un archivo principal marcado en el proyecto». La elección se guarda en el archivo `.enigma_proyecto.json`, dentro del proyecto.

5. Tu primer programa

Vamos a escribir un caso muy corto y a ejecutarlo.

1. Crea un proyecto llamado `hola_detective` con un archivo `hola` (sección 4.2).

2. Escribe este código en el editor:

```
{ Mi primer caso }
caso HolaDetective abierto
    pista detective = expresion(valor: 'Holmes')!
    revelar('Hola, detective', detective)!
caso HolaDetective cerrado!
```

1. Haz clic derecho sobre `hola.eng` y elige **Marcar como archivo principal**.
2. Pulsa **Ejecutar caso** (o `Ctrl+Enter`). El IDE guarda, compila y ejecuta.

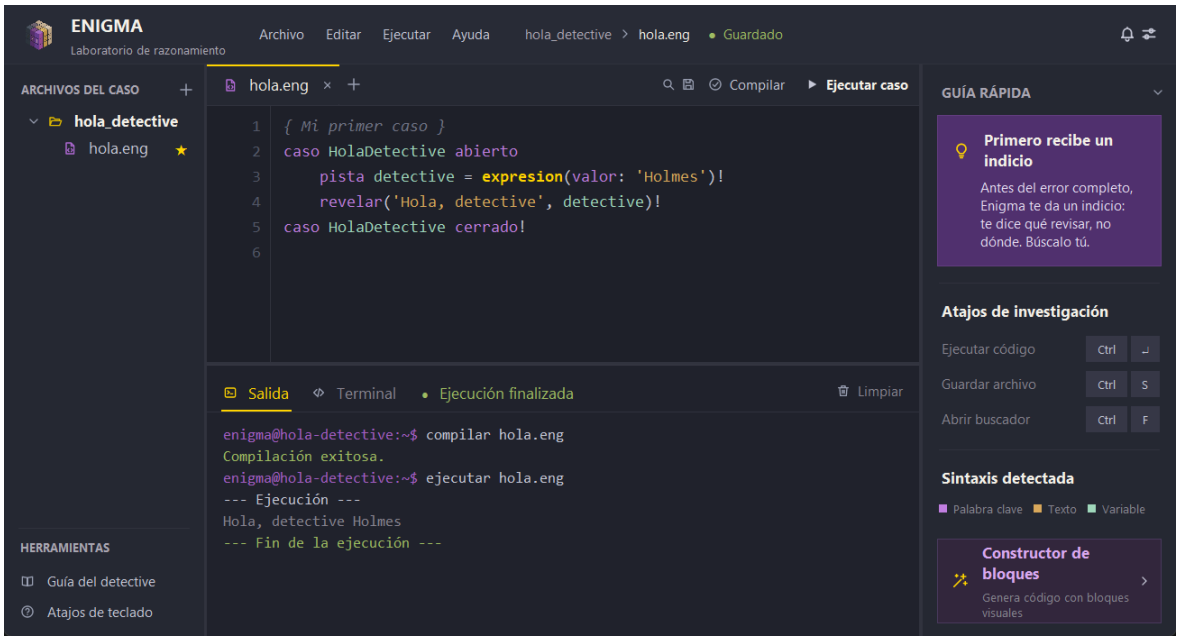


Figura 2. El primer programa ejecutado: la salida aparece en el panel inferior.

Esto es lo que significa cada línea:

Línea	Qué hace
`{ Mi primer caso }`	Un **comentario**: el IDE lo ignora. Los comentarios van entre llaves.
`caso HolaDetective abierto`	Abre el programa principal. Se cierra con `caso HolaDetective cerrado!`; el nombre debe ser el mismo en las dos líneas.
`pista detective = expresion(valor: 'Holmes')!`	Declara una variable llamada `detective` de tipo texto (`expresion`) con el valor «Holmes».
`revelar('Hola, detective', detective)!`	Muestra algo en la salida. Separa lo que le pases con un espacio.
`!`	El **sello**: cierra cada instrucción. Es lo que más se olvida.

6. El lenguaje en pocas páginas

Esta sección es una referencia rápida. La **Guía del detective** del IDE (sección 10) tiene la explicación completa de cada elemento, con ejemplos que puedes insertar en el editor.

6.1 Lo básico

Quiero…	Se escribe así
Declarar una variable	`pista edad = entero(valor: 17)!`
Declarar una constante	`hecho limite = entero(valor: 18)!`
Cambiar el valor de una variable	`edad = conectar(edad, 1)!`
Mostrar algo en la salida	`revelar('Edad:', edad)!`
Pedir un dato al usuario	`consultar(edad)!` (la variable ya debe estar declarada)
Llamar a una escena o un proceso	`pista d = resolver Doble(4)!` o `resolver Saludar('Ana')!`
Importar funciones de otro archivo	`#expediente <analisis.eng>` (sin `!`, al inicio del archivo)
Escribir un comentario	`{ esto lo ignora el IDE }`

6.2 Tipos de datos

Tipo	Para qué sirve	Ejemplo
`entero`	Números sin decimales.	`pista n = entero(valor: 15)!`
`decimal`	Números con decimales.	`pista x = decimal(valor: 15.5)!`
`expresion`	Texto, entre comillas simples.	`pista t = expresion(valor: 'Alex')!`
`declaracion`	Verdadero o falso: `culpable` o `NoCulpable`.	`pista v = declaracion(valor: culpable)!`
`sin_pista`	Ausencia de valor (nulo).	`pista z = entero(valor: sin_pista)!`
`pistas`	Una lista de elementos del mismo tipo.	`pista notas = pistas entero[15, 18, 20]!`
`tablero`	Una matriz de filas y columnas.	`pista celdas = tablero entero[2][2]!`

Se accede a un elemento con corchetes, empezando en cero: `notas[0]` o `celdas[0][1]`.

6.3 Decisiones y ciclos

```
evaluar si (edad >= limite) inicio
    revelar('Mayor de edad')!
fin!
sino inicio
    revelar('Menor de edad')!
fin!
```

Para elegir entre varios casos se usa `suceso` (condición múltiple):

```
suceso (edad) inicio
    1 (edad < 18): revelar('Menor de edad')!
    2 (edad >= 18): revelar('Mayor de edad')!
fin!
```

Para repetir: `rastrear` cuenta de un número a otro (los dos extremos se incluyen) y `seguir mientras` repite mientras se cumpla una condición.

```
rastrear paso desde 1 hasta 3 inicio
    revelar('Paso', paso)!
fin!

seguir mientras (contador <= 3) inicio
    contador = conectar(contador, 1)!
fin!
```

6.4 Escenas y procesos (funciones)

```
escena Doble(n: entero)::entero inicio     { devuelve un valor }
    concluir intensificar(n, 2)!
fin!

proceso Saludar(nombre: expresion) inicio  { no devuelve nada }
    revelar('Hola, ', nombre)!
fin!
```

Los parámetros se escriben `nombre: tipo` y el tipo que devuelve una escena va después de `::`. Una escena **debe** tener al menos un `concluir`.

6.5 Operaciones

Tipo	Operaciones	Ejemplo
Aritméticas	`conectar` (suma) · `descartar` (resta) · `intensificar` (multiplicación) · `repartir` (división, siempre da un decimal)	`conectar(a, b)` `repartir(10, 4)`
Relacionales	`==` igual · `<>` diferente · `>` · `<` · `>=` · `<=`	`edad >= 18`
Lógicas	`y` · `o` · `no(...)` (ocupa toda la condición, ver 6.6)	`(a y b)` `(no(a))`

6.6 Reglas que conviene recordar

- Toda instrucción termina en `!`; también `fin!` y `caso … cerrado!`.
- Los textos van entre **comillas simples** y los comentarios entre **llaves**.
- Cada archivo `.eng` necesita su bloque `caso`, incluso las bibliotecas que solo se importan.
- `no(...)` debe ocupar **toda** la condición: `no(a)` o `no(a y b)` sí funcionan. No se puede mezclar con otras partes: `no(a) y b` y `a y no(b)` dan error. Para combinar, usa `y` y `o` entre condiciones, o comparaciones como `<>`.
- Las **palabras reservadas** no pueden usarse como nombres de variables, constantes ni funciones:

```
pista   hecho   valor   caso   abierto   cerrado   revelar   consultar
evaluar   si   sino   suceso   inicio   fin   rastrear   desde   hasta
seguir   mientras   escena   proceso   resolver   concluir   conectar
descartar   intensificar   repartir   o   y   no   entero   decimal
expresion   declaracion   pistas   tablero   culpable   NoCulpable
sin_pista
```

7. Compilar y ejecutar

Botón	Qué hace
Compilar	Revisa el código **sin ejecutarlo**: busca errores de escritura, de estructura, de tipos y de importaciones, y calcula las advertencias. Si todo está bien dice «Compilación exitosa».
Ejecutar caso (`Ctrl+Enter`)	Compila y, **solo si no hay errores**, ejecuta el programa y muestra lo que imprime. Siempre empieza por el archivo principal.

Los dos guardan antes los archivos que tengas abiertos, así que no necesitas guardar a mano.

7.1 Pedir datos con consultar

Cuando el programa llega a un `consultar`, el panel de salida **se queda esperando**: escribe el dato directamente ahí y pulsa `Enter`.

```
pista edad = entero(valor: 0)!
revelar('Escribe tu edad:')!
consultar(edad)!
revelar('Tu edad es:', edad)!
```

Si el dato no corresponde al tipo de la variable (por ejemplo, texto para un `entero`), el IDE lo avisa.

7.2 Salida y Terminal

El panel inferior tiene dos solapas: **Salida**, con el resultado de lo último que hiciste, y **Terminal**, que guarda una bitácora con la hora de cada compilación y ejecución. El botón **Limpiar** vacía la solapa activa.

7.3 Atajos de teclado

Atajo	Acción
`Ctrl+Enter`	Compilar y ejecutar
`Ctrl+S`	Guardar el archivo
`Ctrl+O`	Abrir una carpeta
`Ctrl+F`	Buscar en el archivo
`Ctrl+W`	Cerrar la pestaña
`F1`	Mostrar u ocultar la Guía del detective

8. Entender los mensajes

Lo más particular de Enigma es cómo te avisa de los errores. En lugar de decirte todo de golpe, lo hace en **dos pasos**, para que practiques el razonamiento.

Color	Qué es
Verde	Todo salió bien: compilación exitosa o ejecución finalizada.
Naranja	Un **indicio**: te orienta sobre el error sin decir dónde está.
Rojo	La **explicación completa** de un error.
Azul	Una **advertencia**: algo sospechoso que no impide ejecutar.

8.1 Los dos pasos de un error

Paso 1. La primera vez que compilas un código con un error, solo ves un indicio. Fíjate en que no dice la línea ni el nombre: te dice qué tipo de problema revisar.

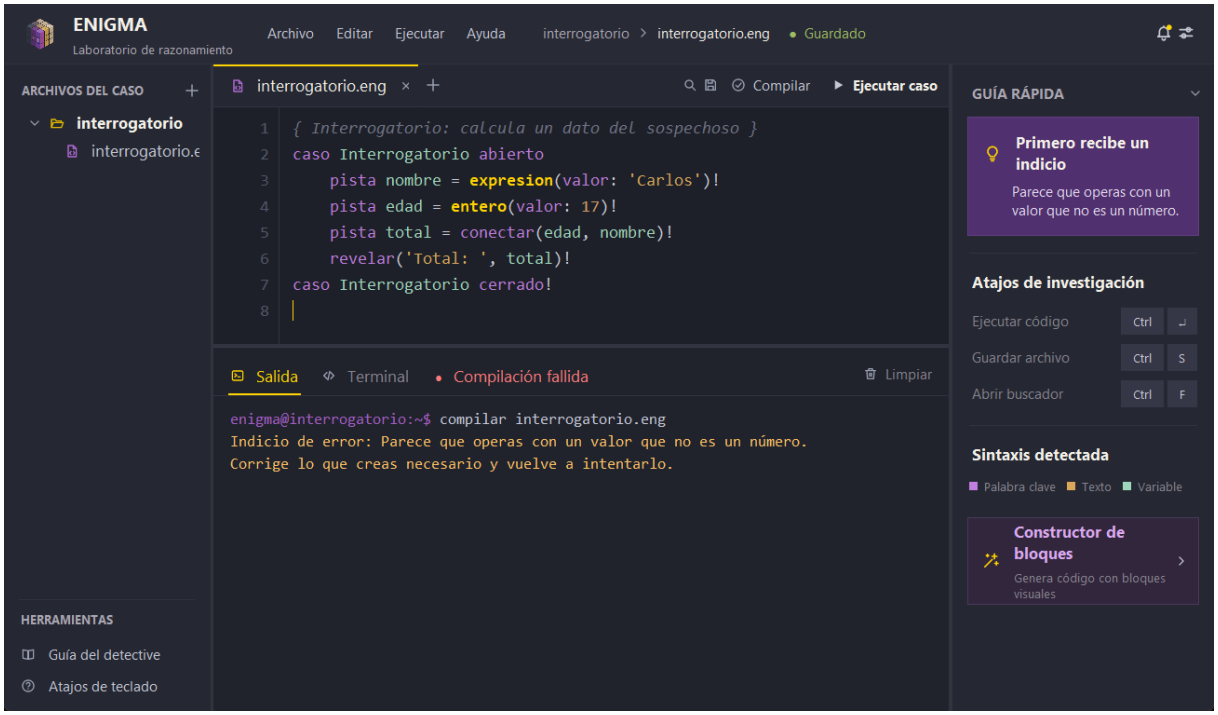


Figura 3. Primer paso: solo el indicio, en naranja.

Paso 2. Si vuelves a compilar **sin haber corregido ese error**, el IDE entiende que necesitas ayuda y muestra la explicación completa: tipo de error, archivo, línea, descripción y posible causa.

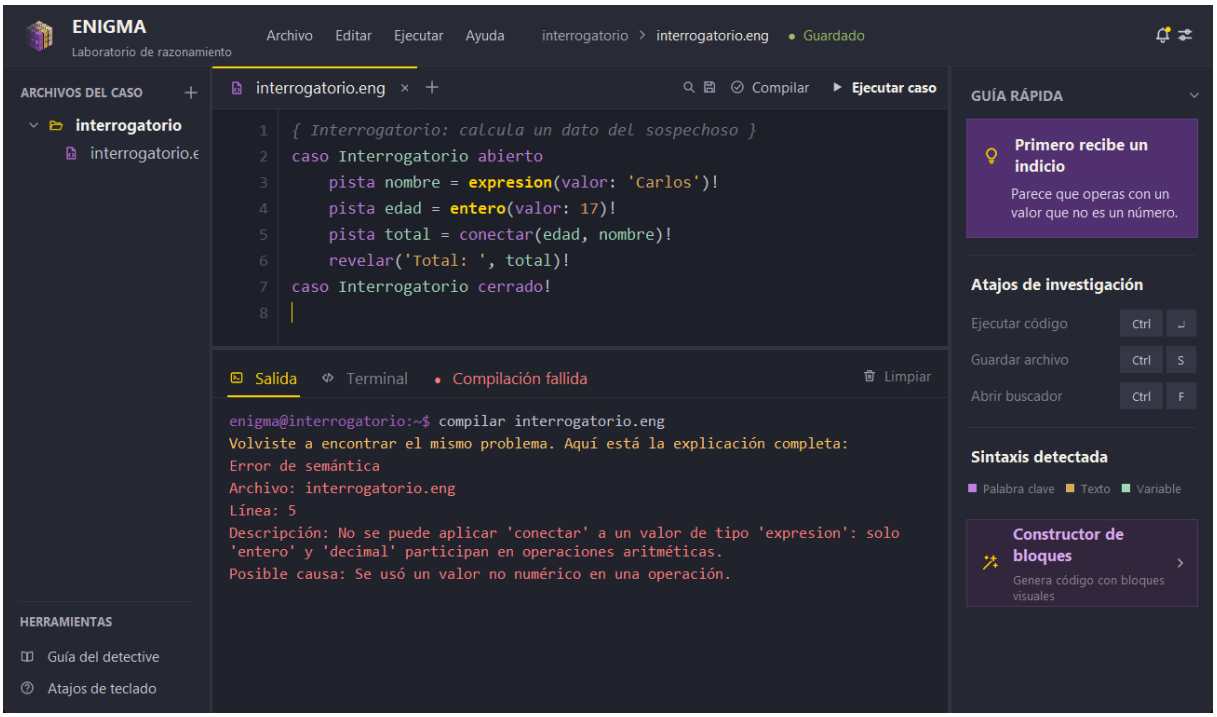


Figura 4. Segundo paso: la explicación completa, en rojo.

El indicio más reciente también queda en la tarjeta «Primero recibe un indicio» de la Guía rápida.

8.2 Tipos de error

Tipo	Cuándo ocurre	Ejemplos
Léxico	El texto tiene algo que Enigma no reconoce.	Un símbolo raro, un comentario o un texto sin cerrar.
Sintáctico	La estructura está mal escrita.	Falta un `!`, un bloque sin `fin!`, un paréntesis sin cerrar.
Semántico	Está bien escrito pero no tiene sentido.	Variable sin declarar, tipos que no combinan, cambiar un `hecho`, una escena sin `concluir`.
Importación	Hay un problema con los archivos importados.	El archivo no existe, importación circular, función repetida.
Ejecución	Solo se descubre al correr el programa.	Dividir entre cero, un índice fuera de la lista.

Los errores de **ejecución** dependen de los valores reales, por eso **Compilar** no puede encontrarlos: aparecen al pulsar **Ejecutar caso**.

8.3 Advertencias

Las advertencias avisan de código válido pero sospechoso. **No impiden compilar ni ejecutar** y también aparecen cuando hay un error. Siguen la misma idea de dos pasos: primero un resumen sin decir dónde y, si compilas de nuevo sin cambiar nada, el detalle con archivo y línea.

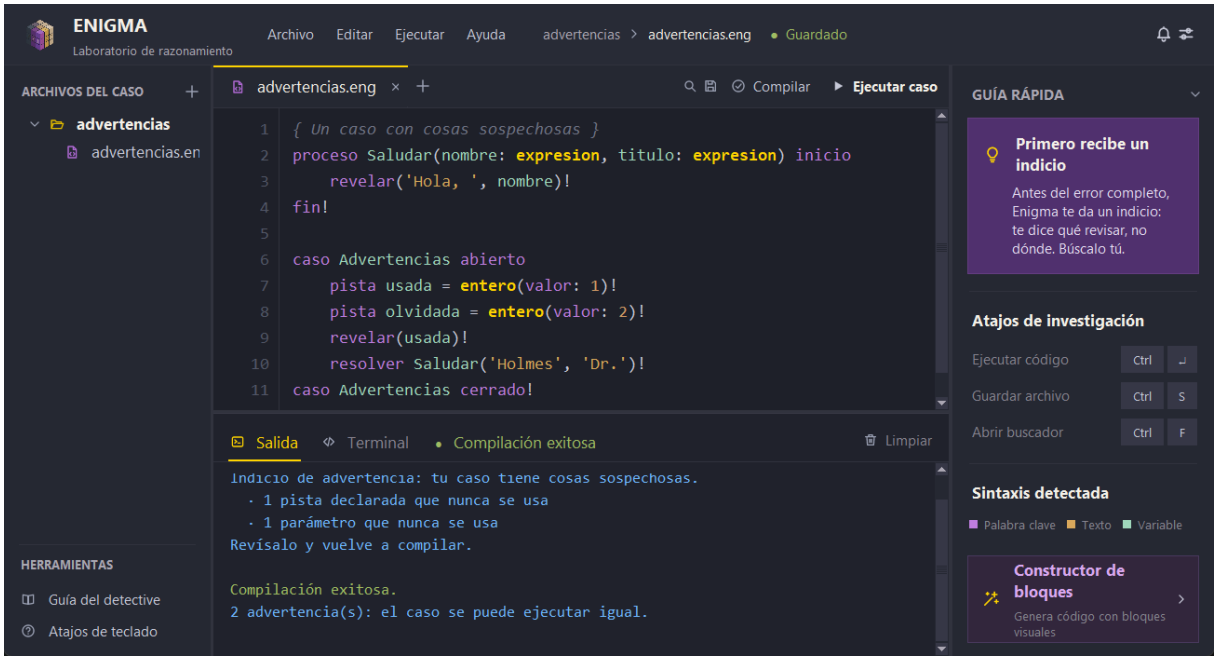


Figura 5. Advertencias en azul: el caso compila y se puede ejecutar.

Advertencia	Cuándo aparece
Pista que nunca se usa	Declaraste una variable y nunca la usaste.
Parámetro que nunca se usa	Una escena o un proceso no usa uno de sus parámetros.
Código que nunca se ejecuta	Hay instrucciones después de un `concluir` en el mismo bloque.

9. Importar funciones entre archivos

Un proyecto puede repartirse en varios archivos. Un archivo puede usar las escenas y los procesos de otro con la directiva `#expediente`, escrita **al inicio** del archivo y **sin `!`**:

```
#expediente <analisis.eng>
```

- Se busca el archivo **junto al archivo que lo importa**. Para una subcarpeta: `#expediente <utilidades/pistas.eng>`.
- Al importar, las escenas y los procesos del otro archivo quedan disponibles para usarlos con `resolver`.
- **Cada archivo necesita su propio bloque `caso`**, también la biblioteca.
- El IDE detecta archivos que no existen, funciones repetidas e importaciones circulares (A importa B y B importa A).
- Si hay un error dentro del archivo importado, el mensaje indica **ese archivo** y su línea.

El proyecto `ejemplos/investigacion_caso` tiene dos archivos: `principal.eng` importa `analisis.eng` y le pregunta con `resolver EsMayorDeEdad(edad)` si el sospechoso es mayor de edad.

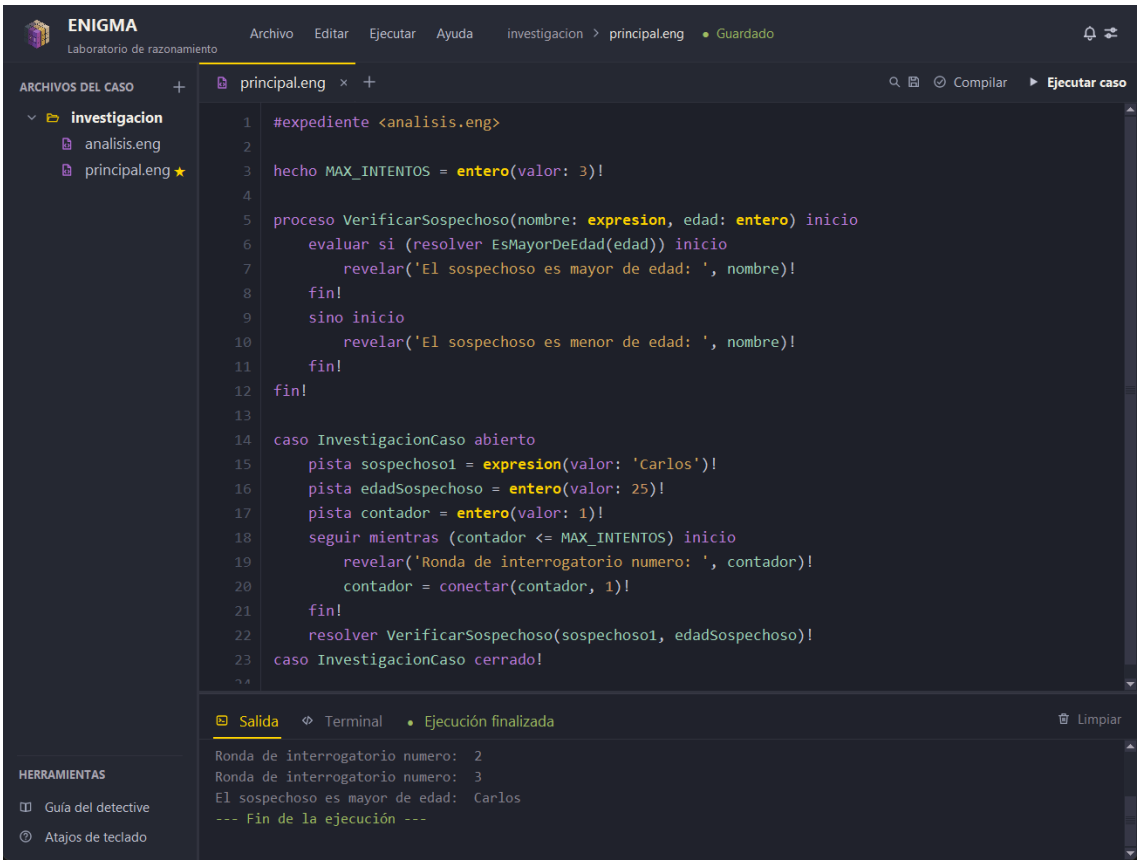


Figura 6. Un proyecto con dos archivos: el principal importa la biblioteca analisis.eng.

10. Las ayudas del IDE

10.1 La Guía del detective

Es la consulta del lenguaje dentro del IDE. Se abre con `F1`, con el botón **Guía del detective** de la esquina inferior izquierda o desde el menú **Ayuda**. Tiene una sección para cada tema: palabras reservadas, sintaxis, estructuras de control, funciones, operaciones, semántica, tipos de datos, importaciones, ejemplos, casos de práctica y atajos.

- Cambia de sección con las flechas **‹ ›** o con el menú desplegable del título.

- Cada ejemplo de código tiene el botón **Insertar en el editor**, que **crea un proyecto y un archivo nuevos** con ese código (te pide el nombre y dónde guardarlo), y el botón **Copiar**.

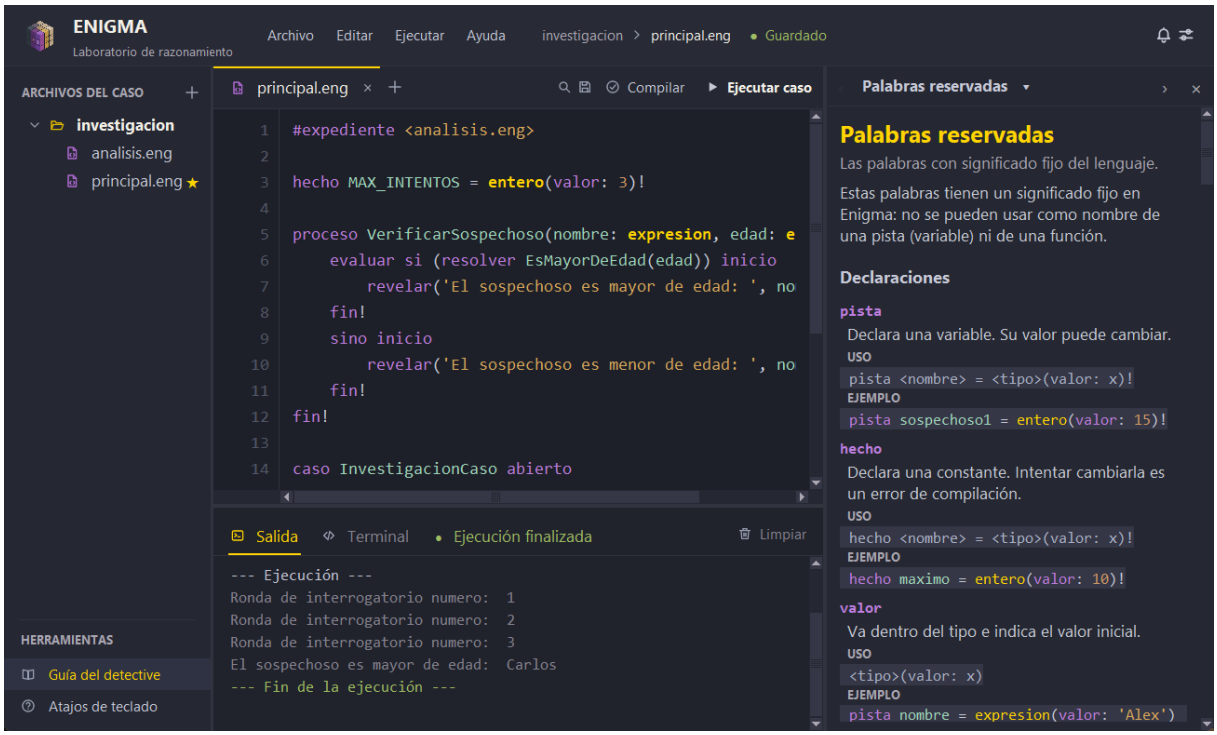


Figura 7. La Guía del detective abierta junto al editor.

10.2 Casos de práctica y acertijos

En la sección **Casos de práctica** hay retos con un enunciado y un código de inicio, **sin la solución**: tú escribes la parte que falta. Los **acertijos** son casos completos que se ejecutan. Ambos se insertan en un proyecto nuevo con **Insertar en el editor**.

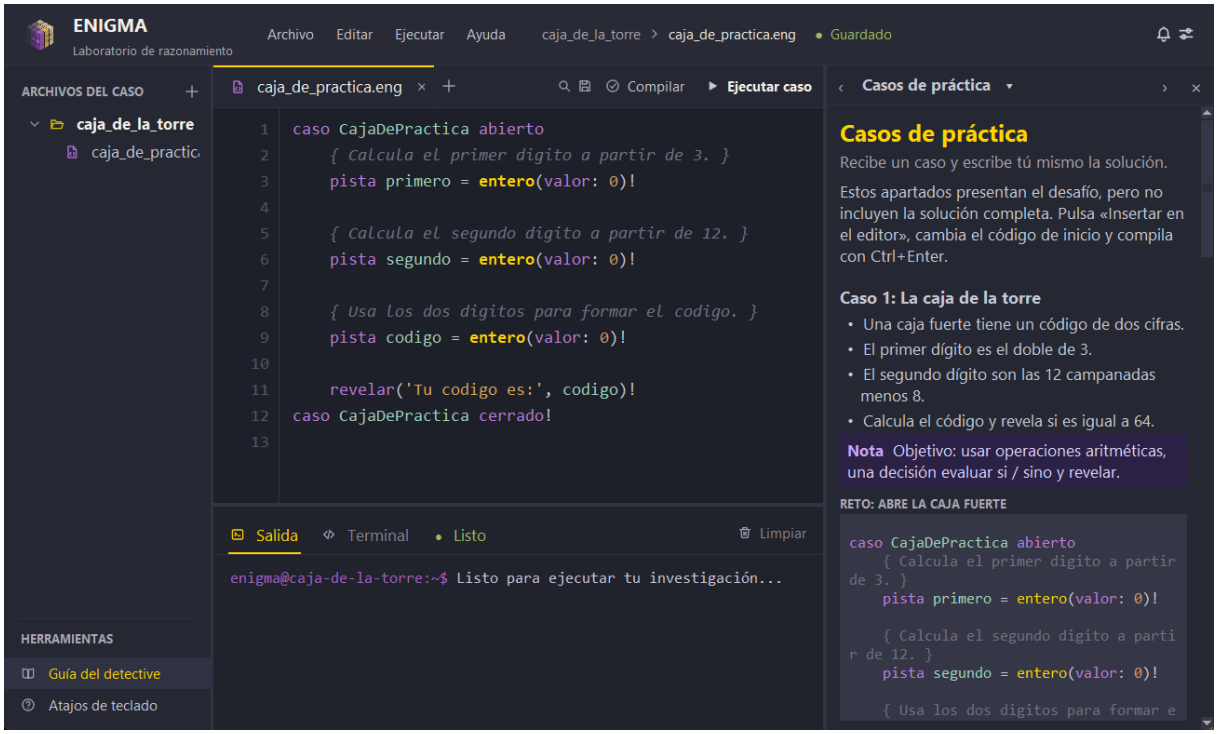


Figura 8. Un caso de práctica abierto en el editor.

10.3 El Constructor de bloques

Arma instrucciones con formularios, sin escribir la sintaxis. Se abre desde **Ayuda → Constructor de bloques** o con la tarjeta morada de la Guía rápida.

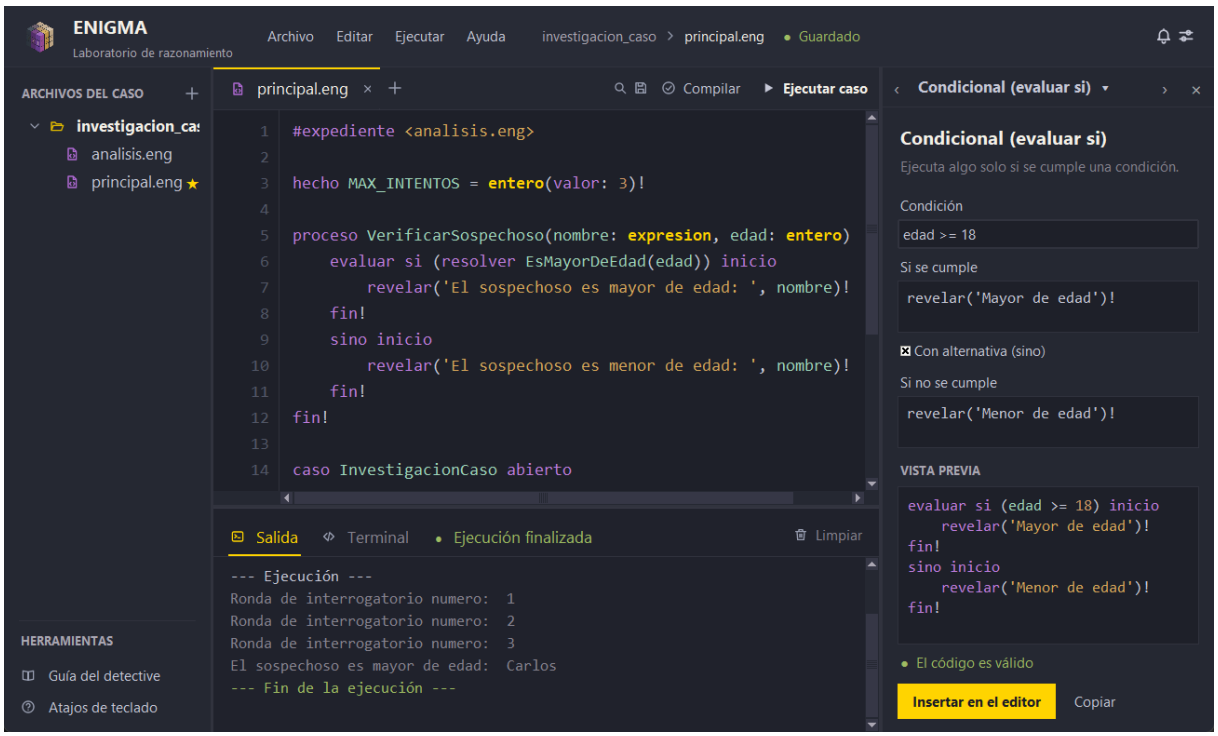


Figura 9. El Constructor de bloques con el bloque «Condicional».

1. Elige un bloque con las flechas ‹ › o con el menú desplegable (▾).
2. Completa los campos del formulario. La **vista previa** muestra el código y avisa si es válido.
3. Pulsa **Insertar en el editor**. Si tienes un archivo abierto, el bloque se escribe donde esté el cursor y puedes ir sumando bloques; si no hay ninguno, crea un proyecto y un archivo nuevos. **Copiar** lo lleva al portapapeles.

Los bloques disponibles son: Declarar una pista, Mostrar un mensaje, Preguntar un dato, Cambiar una pista, Condicional (`evaluar si`), Ciclo contado (`rastrear`), Ciclo con condición (`seguir mientras`), Función con retorno (`escena`), Función sin retorno (`proceso`), Importar otro archivo y Esqueleto de un caso.

10.4 Buscar y preferencias

- `Ctrl+F` abre la barra de búsqueda del archivo activo.
- **Editar → Preferencias…** (o el ícono de ajustes) cambia el tamaño de la letra del editor, de 9 a 20.

11. Proyectos de ejemplo

El proyecto trae varios ejemplos para abrir y probar. Se abren con `python main.py <carpeta>` o con **Archivo → Abrir carpeta…**.

Carpeta	Qué muestra
`ejemplo_gui`	Un programa corto con una escena, un condicional y un ciclo.
`ejemplos/caso_promedio`	Una escena que calcula el promedio de dos notas.
`ejemplos/acertijo_caja_fuerte`	Acertijo: calcular el código de dos cifras de una caja fuerte.
`ejemplos/acertijo_sospechosos`	Acertijo: deducir quién robó una obra usando coartadas y huellas.
`ejemplos/investigacion_caso`	**Proyecto de dos archivos** con importaciones. Ya trae marcado su archivo principal.

> **Recuerda.** Salvo `investigacion_caso`, los ejemplos son de un solo archivo y **no traen archivo principal marcado**: haz clic derecho sobre el archivo y elige **Marcar como archivo principal** antes de ejecutarlo.

12. Solución de problemas

Lo que ves	Qué significa	Qué hacer
«No hay un archivo principal marcado en el proyecto»	El proyecto no sabe por dónde empezar.	Clic derecho sobre el archivo → **Marcar como archivo principal**.
Indicio: «Parece que falta algo al final de alguna instrucción»	Casi seguro falta un `!`.	Revisa que cada instrucción, cada `fin` y cada `caso … cerrado` termine en `!`.
«Parece que usaste una palabra reservada como nombre»	Usaste `y`, `o`, `no`, `si`, `valor`… como nombre.	Cambia el nombre. La lista de palabras reservadas está en la sección 6.6.
«Parece que usas algo que no declaraste»	Un nombre mal escrito o una variable sin declarar.	Declárala con `pista` o `hecho`, o corrige la ortografía.
«Parece que un archivo que importas no está donde se esperaba»	El `#expediente` apunta a un archivo que no existe.	Revisa el nombre y que esté en la misma carpeta (o la subcarpeta indicada).
«Parece que una escena no entrega el valor que promete»	La escena no tiene ningún `concluir`.	Agrégalo con el valor que debe devolver, o conviértela en `proceso`.
«Parece que una división no se puede calcular con esos valores»	Un `repartir` con divisor cero.	Revisa los valores antes de dividir.
«Parece que falta algo en alguna parte de tu código» al compilar un archivo importado	Un archivo `.eng` sin su bloque `caso`.	Agrega `caso Nombre abierto` … `caso Nombre cerrado!`.
Un error al mezclar `no(...)` con `y` u `o`, por ejemplo `no(a) y b`	`no(...)` debe ocupar toda la condición.	Escribe `no(a y b)`, o usa comparaciones como `<>`.
Cambié el código y el resultado no cambió	Estás ejecutando el archivo principal, no el que editas.	Marca como principal (★) el archivo correcto.
El IDE no abre en Linux	Falta tkinter.	`sudo apt install python3-tk`